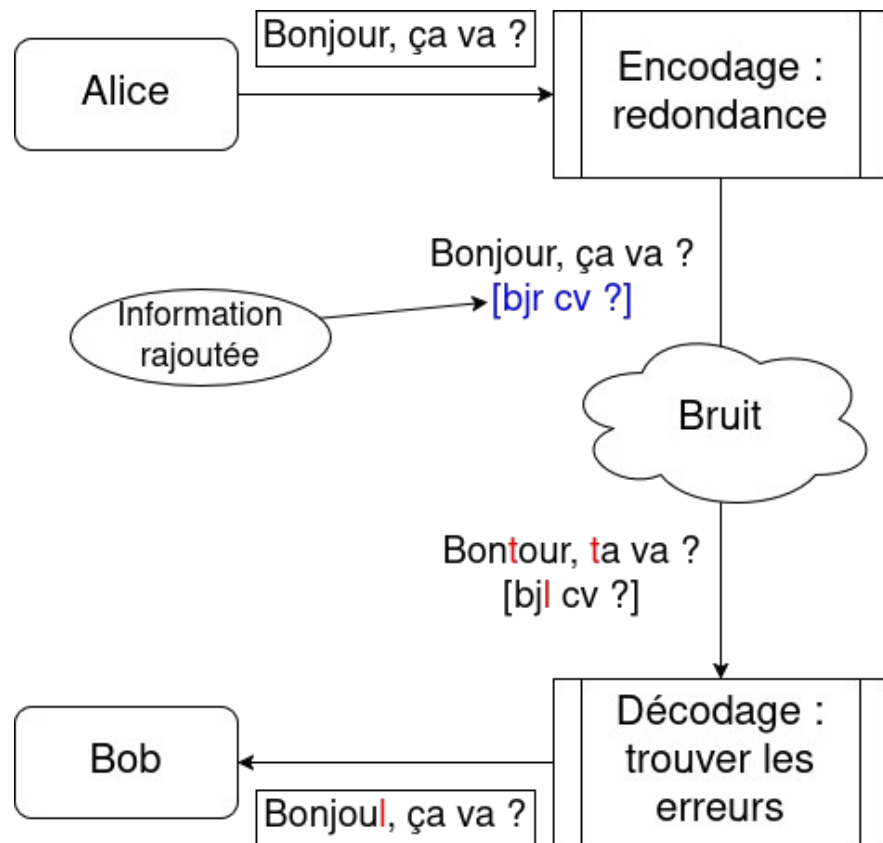


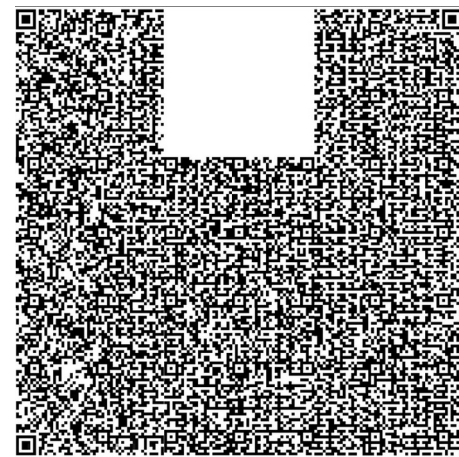
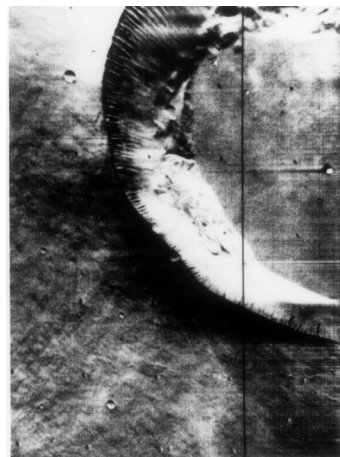
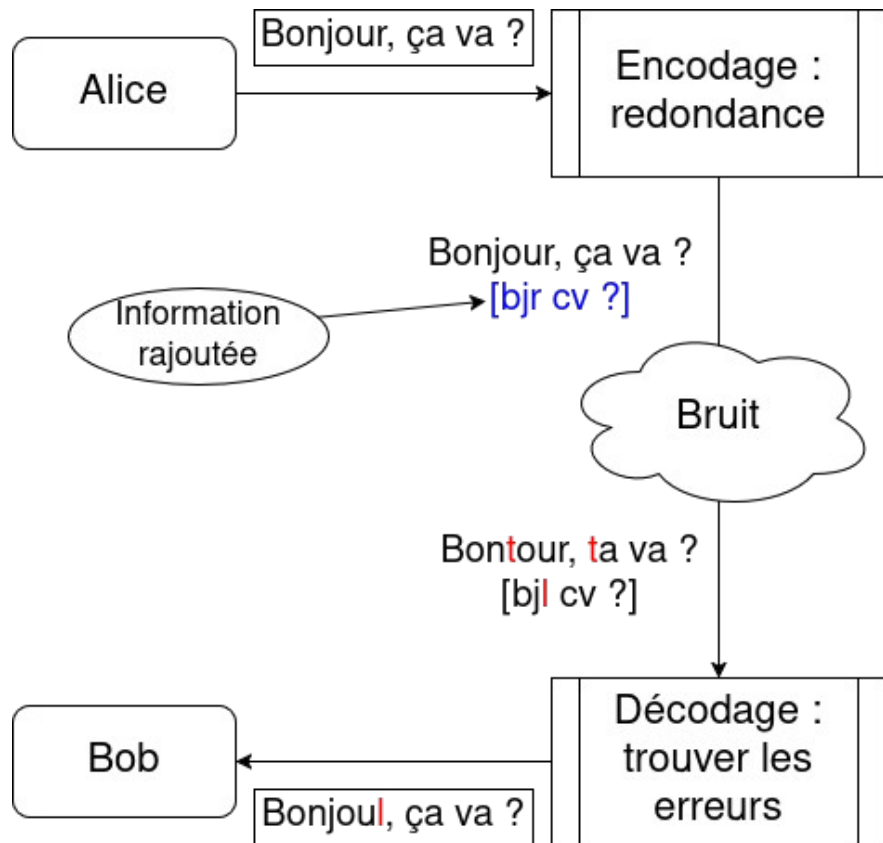
Codes cycliques et Reed-Solomon

Philéas Lesourd, Vincent Rubin

Introduction



Introduction

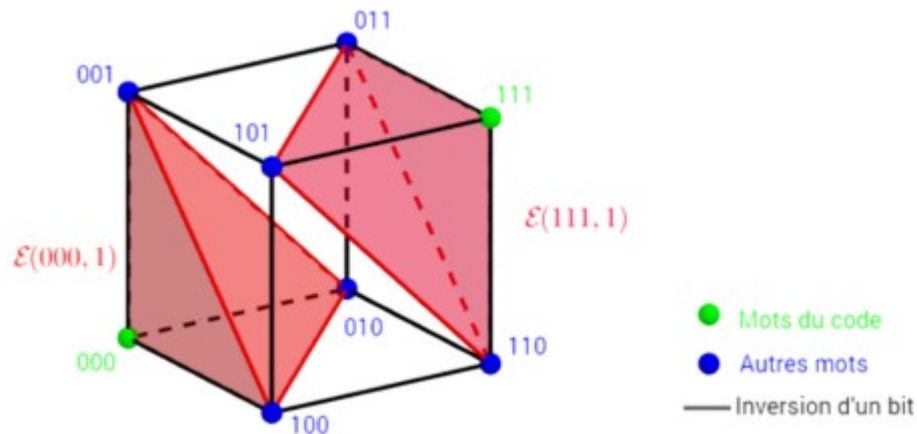


Comment garantir l'intégrité d'un message et le corriger ?

- 1) Le code cyclique original de Reed-Solomon
- 2) Gorenstein-Zierler
 - a. Encodage
 - b. Polynôme d'erreur
 - c. Décodage
- 3) Résultats
- 4) Complexités et paramètres

1) Le code cyclique original de Reed-Solomon

- Code correcteur :
 - technique qui sert à assurer une *transmission* par un canal qui pourrait la *détériorer*.



1) Le code cyclique original de Reed-Solomon

- Code linéaire :
 - *espaces vectoriels* de dimension finie sur des *corps finis*.
 - applications injectives d'un espace vectoriel dans un autre de dimension *plus grande*.

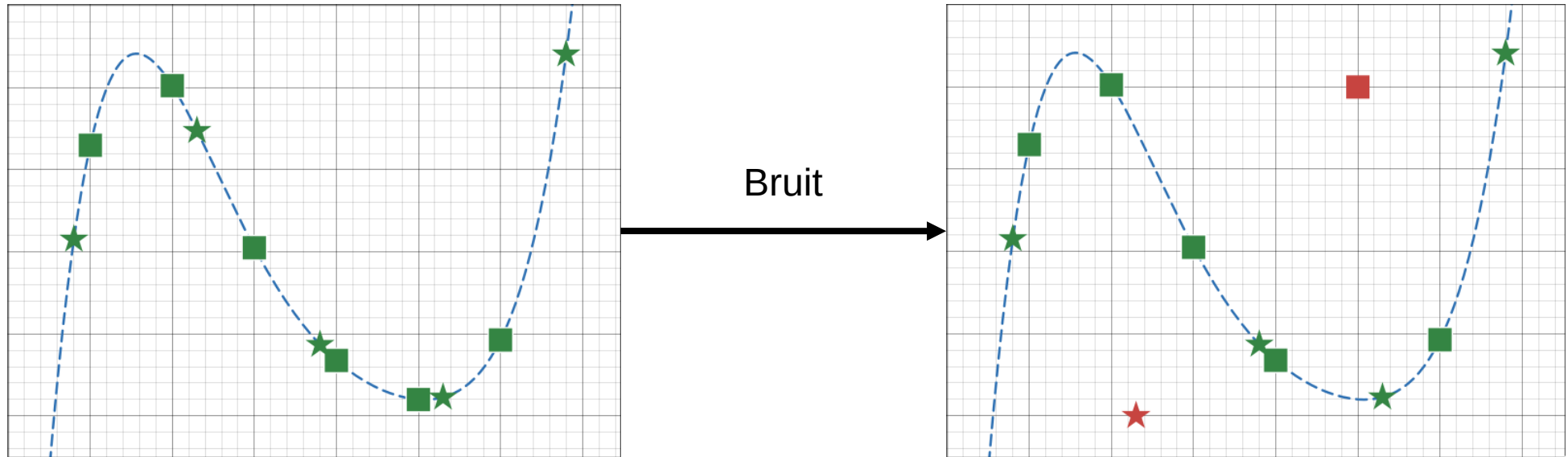
1) Le code cyclique original de Reed-Solomon

- **Code linéaire :**
 - *espaces vectoriels* de dimension finie sur des *corps finis*.
 - applications injectives d'un espace vectoriel dans un autre de dimension *plus grande*.
- **Code cyclique :**
 - code linéaire
 - Si $(c_1, \dots, c_n)$ est un mot encodé, $(c_2, \dots, c_n, c_1)$ en est un.

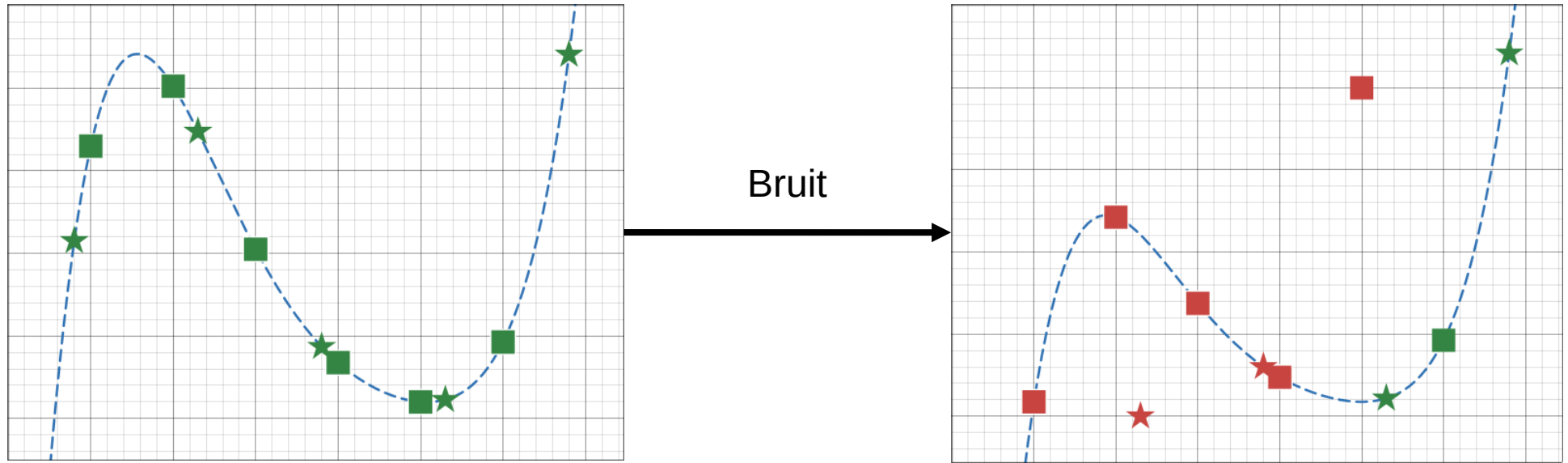
1) Le code cyclique original de Reed-Solomon

- On utilise $\mathbb{F}_{256} = \mathbb{F}_2[X]/(Q)$ avec Q irréductible sur $\mathbb{F}_2[X]$
- Les « mots » sont alors dans $\mathbb{F}_{256}[X]$
- $\mathbb{F}_{256}^*$ est un groupe cyclique. On note $\alpha = \overline{X}$ qui engendre $\mathbb{F}_{256}^*$

1) Le code cyclique original de Reed-Solomon



1) Le code cyclique original de Reed-Solomon



2)a. Gorenstein-Zierler : encodage

Message de longueur k = $A \in \mathbb{F}_{256}[X]$

$$G(X) = \prod_{j=1}^{2t} (X - \alpha^j) \quad \text{avec } 2t + k \leq 255$$

$$AX^{2t} = GQ + B$$

On encode A par : $C = \overbrace{AX^{2t}}^{k-1 \text{ coeffs}} - \overbrace{B}^{\text{info rajoutée}} = GQ.$

$$\forall i \in \llbracket 1, 2t \rrbracket, C(\alpha^i) = 0$$

2)b. Gorenstein-Zierler : polynôme d'erreur

Polynôme d'erreur : $E = D - C = \sum_{r=1}^{n_e} y_r X^{i_r}$

 connu  inconnu

On a $n_e \leq t$ erreurs aux positions i_r .

$$\forall j \in \llbracket 1, 2t \rrbracket, S_j = D(\alpha^j) = E(\alpha^j) = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

 connus  inconnus  inconnus

2)c. Gorenstein-Zierler : décodage

1 On calcule les i_r :

$$\Lambda(X) = \prod_{r=1}^{n_e} (1 - \alpha^{i_r} X) = 1 + \lambda_1 X + \lambda_2 X^2 + \dots + \lambda_{n_e} X^{n_e}$$

$$S_j = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

2)c. Gorenstein-Zierler : décodage

1 On calcule les i_r :

$$S_j = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

$$\Lambda(X) = \prod_{r=1}^{n_e} (1 - \alpha^{i_r} X) = 1 + \lambda_1 X + \lambda_2 X^2 + \dots + \lambda_{n_e} X^{n_e}$$

$$\begin{aligned} \sum_{i=1}^{\boxed{\nu}} S_{\nu+j-i} \lambda_i &= \sum_{i=1}^{\nu} \left(\sum_{r=1}^{\nu} y_r (\alpha^{i_r})^{\nu+j-i} \right) \lambda_i = \sum_{r=1}^{\nu} y_r (\alpha^{i_r})^{\nu+j} \sum_{i=1}^{\nu} (\alpha^{i_r})^{-i} \lambda_i \\ &= \sum_{r=1}^{\nu} y_r (\alpha^{i_r})^{\nu+j} (\Lambda(\alpha^{-i_r}) - 1) = - \sum_{r=1}^{\nu} y_r (\alpha^{i_r})^{\nu+j} \\ &= -S_{\nu+j} \end{aligned}$$

2)c. Gorenstein-Zierler : décodage

1 On calcule les i_r :

$$S_j = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

Pour ν allant de t à 1 (dans l'ordre décroissant) :

$$\begin{pmatrix} S_\nu & S_{\nu-1} & \dots & S_1 \\ S_{\nu+1} & S_\nu & \dots & S_2 \\ \vdots & \vdots & \ddots & \vdots \\ S_{2\nu-1} & S_{2\nu-2} & \dots & S_\nu \end{pmatrix} \begin{pmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_\nu \end{pmatrix} = - \begin{pmatrix} S_{\nu+1} \\ S_{\nu+2} \\ \vdots \\ S_{2\nu} \end{pmatrix}$$

2)c. Gorenstein-Zierler : décodage

2 On calcule les y_r :

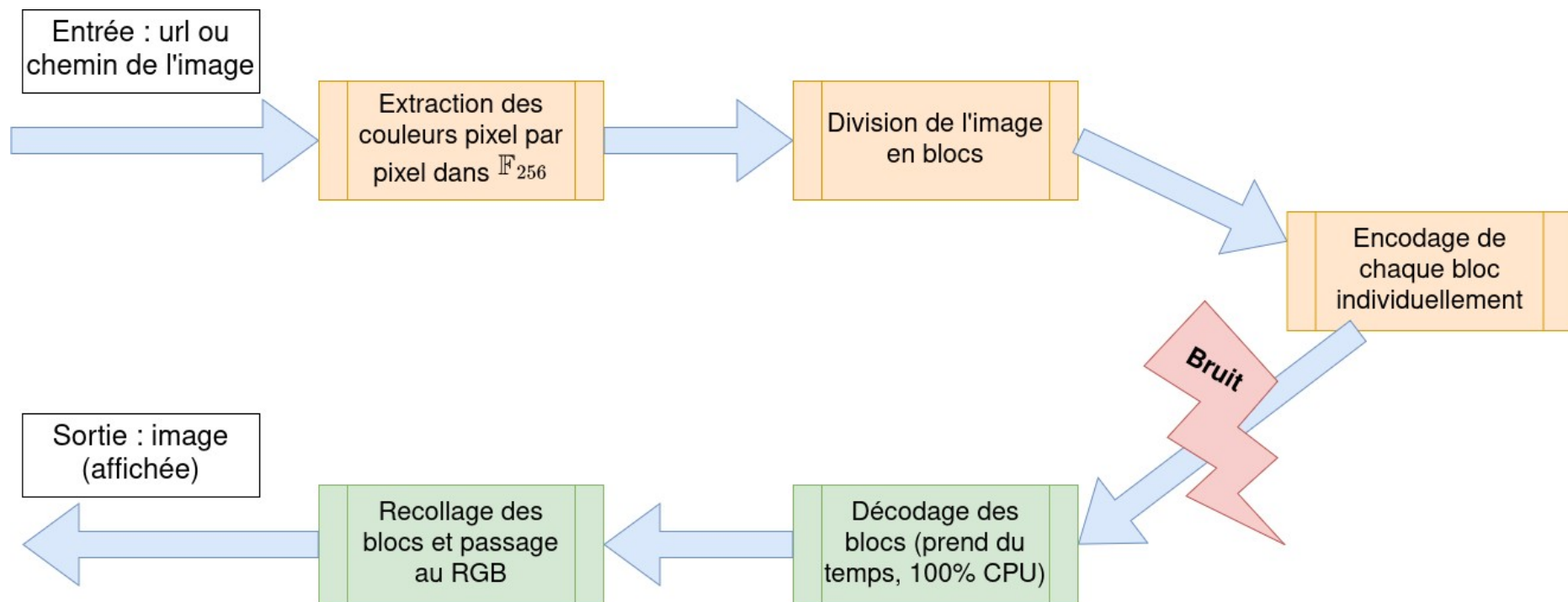
$$\begin{pmatrix} \alpha^{i_1} & \alpha^{i_2} & \dots & \alpha^{i_{n_e}} \\ (\alpha^{i_1})^2 & (\alpha^{i_2})^2 & \dots & (\alpha^{i_{n_e}})^2 \\ \vdots & \vdots & \ddots & \vdots \\ (\alpha^{i_1})^{n_e} & (\alpha^{i_2})^{n_e} & \dots & (\alpha^{i_{n_e}})^{n_e} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n_e} \end{pmatrix} = - \begin{pmatrix} S_1 \\ S_2 \\ \vdots \\ S_{n_e} \end{pmatrix}$$

$$E = D - C = \sum_{r=1}^{n_e} y_r X^{i_r}$$

A est dans les k premiers coefficients de C .

$$S_j = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

3) Résultats



3) Résultats



originale

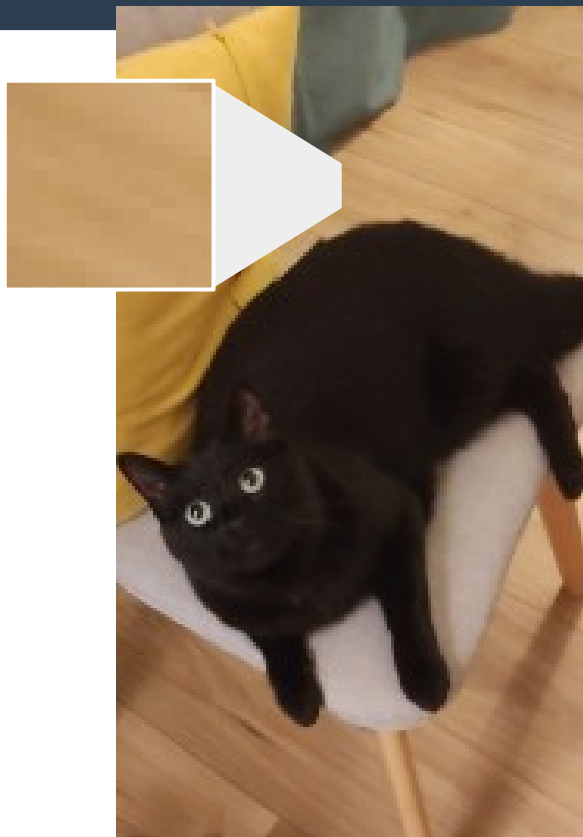


bruit à 6 %

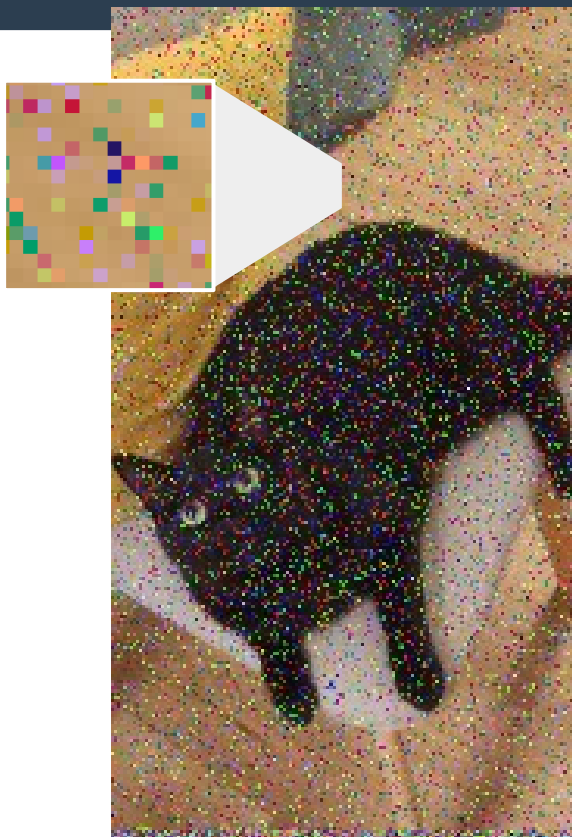


blocs de 12x18, $t = 19(19)$
 $e = 0,045 \%$

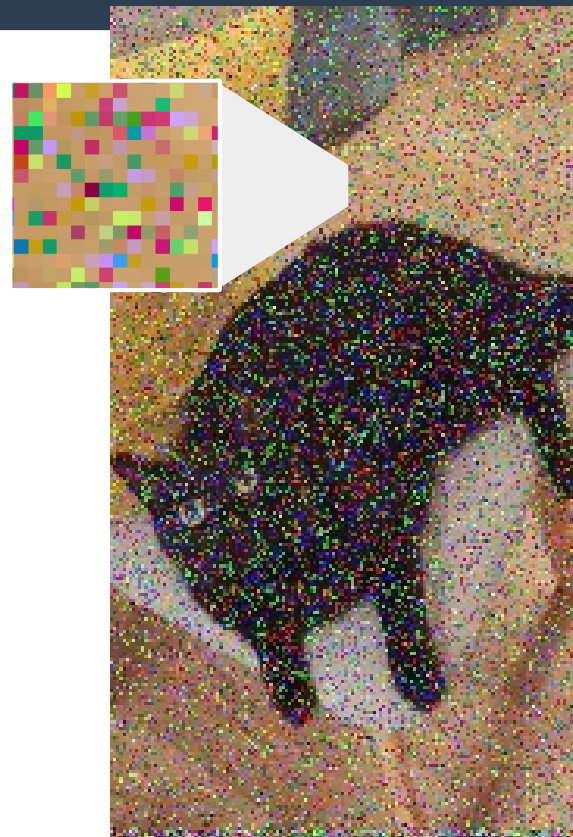
3) Résultats



originale



bruit à 15 %



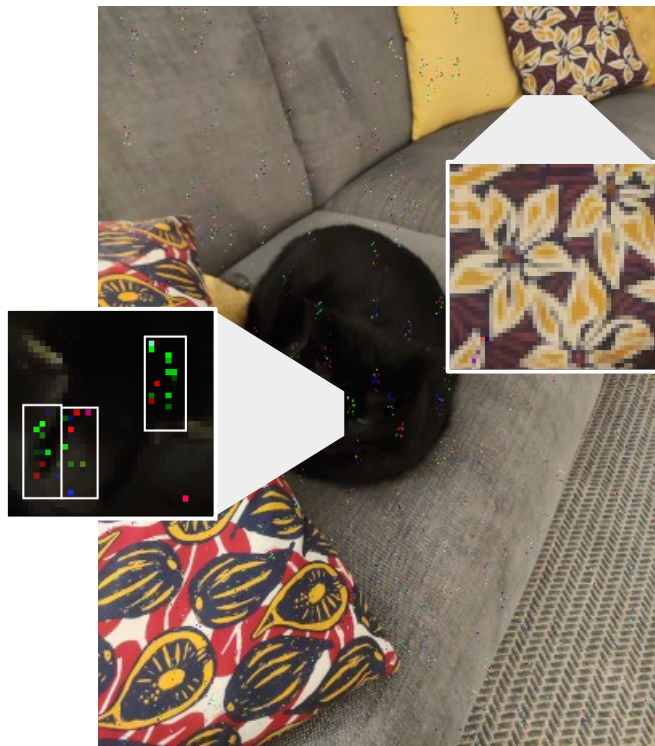
bruit à 25 %

3) Résultats

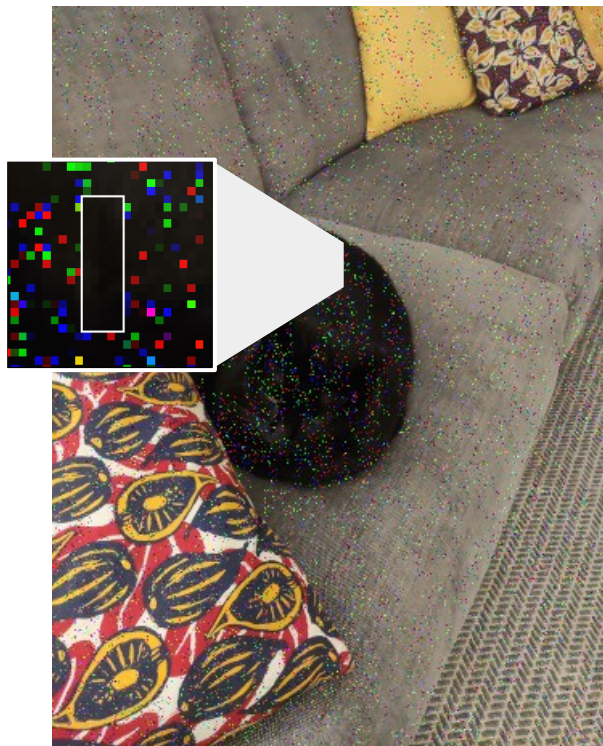
Temps de décodage :

14 s

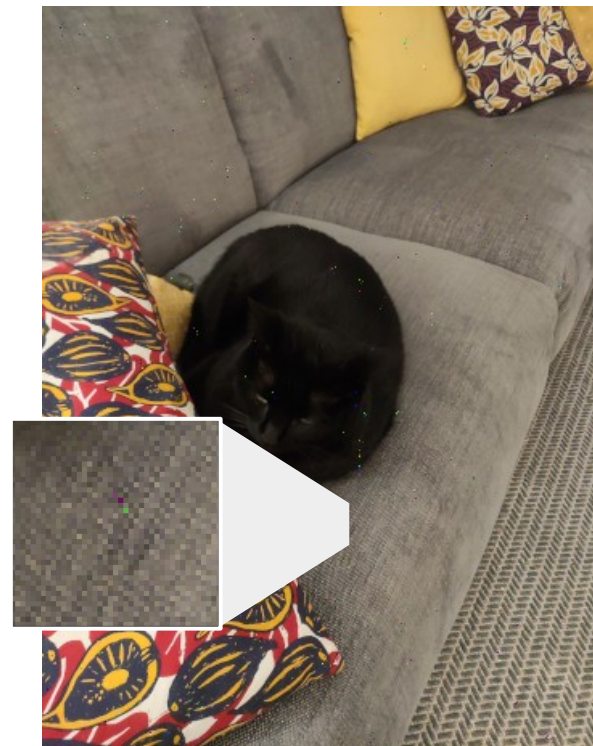
695 s



bruit 15 %, blocs de 15x15,
 $t = 15$ (/15), $e = 2,16$ %

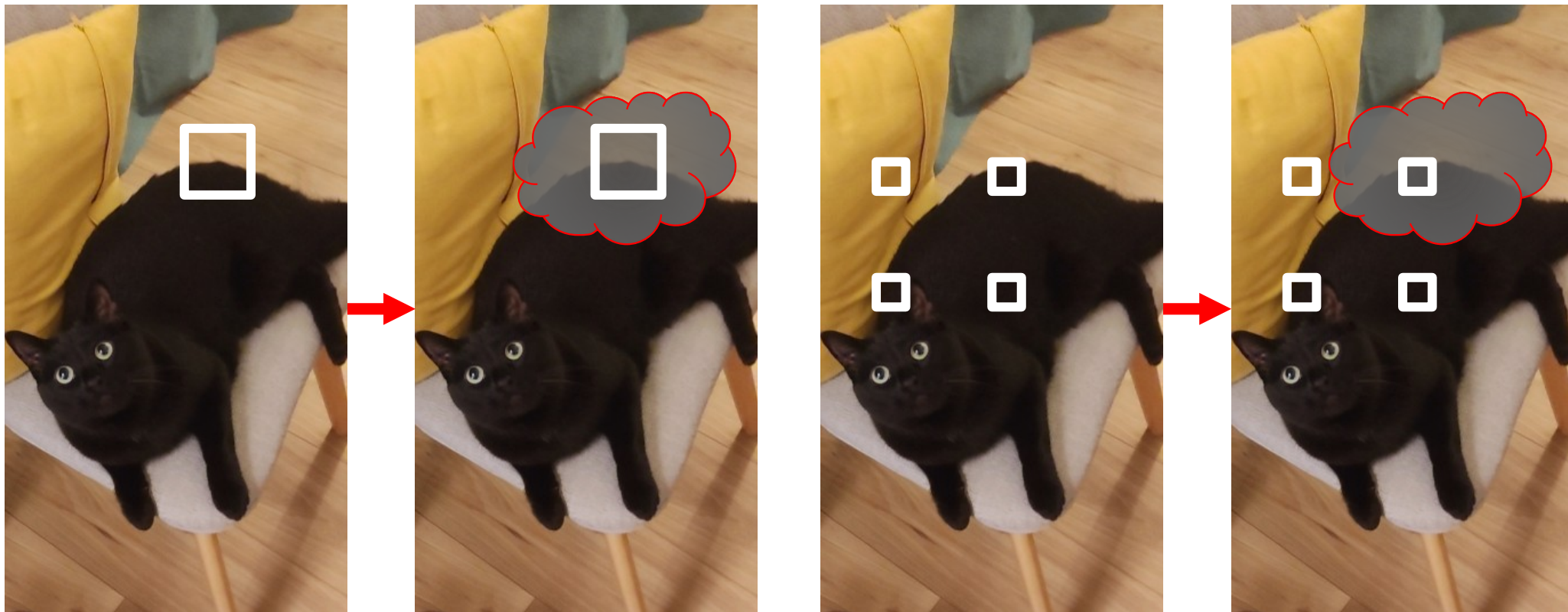


bruit 25 %, blocs de 15x15,
 $t = 15$ (/15), $e = 17,4$ %



bruit 25 %, blocs de 10x10,
 $t = 60$ (/70), $e = 0,42$ %

3) Résultats : précisions

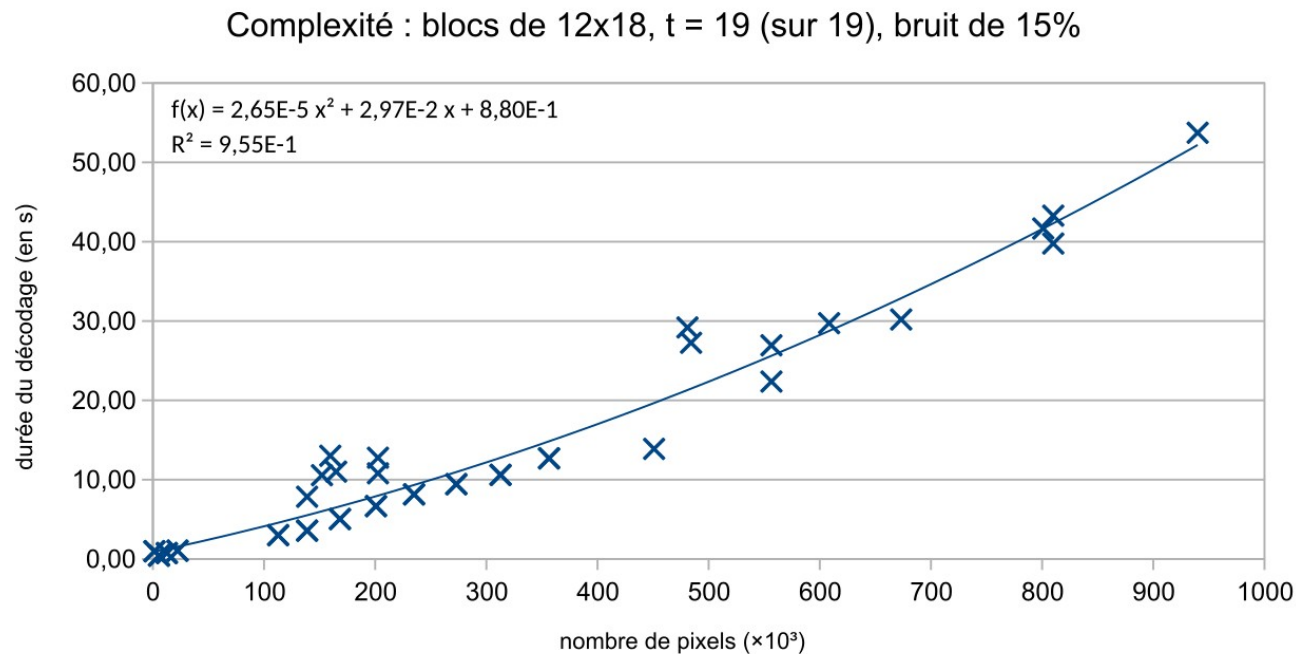


4) Complexités et paramètres

Complexité en :

$$O(x^2)$$

x = nombre de
pixels



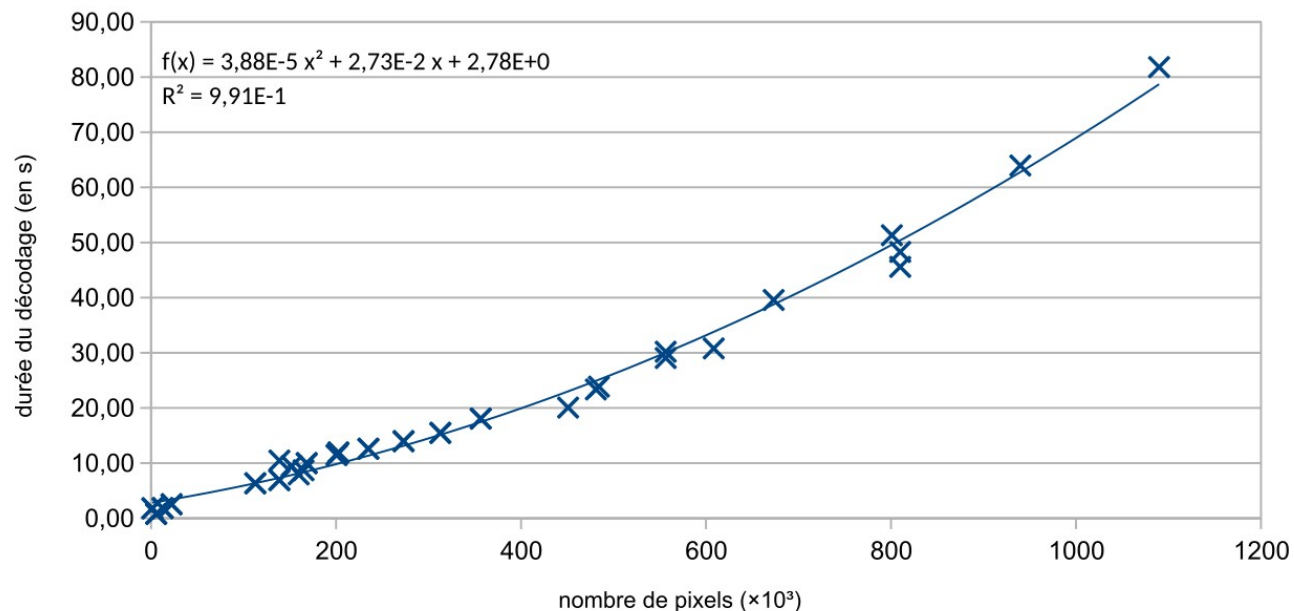
4) Complexités et paramètres

Complexité en :

$$O(x^2)$$

x = nombre de
pixels

Complexité : blocs de 12x18, $t = 19$ (sur 19), bruit de 4%

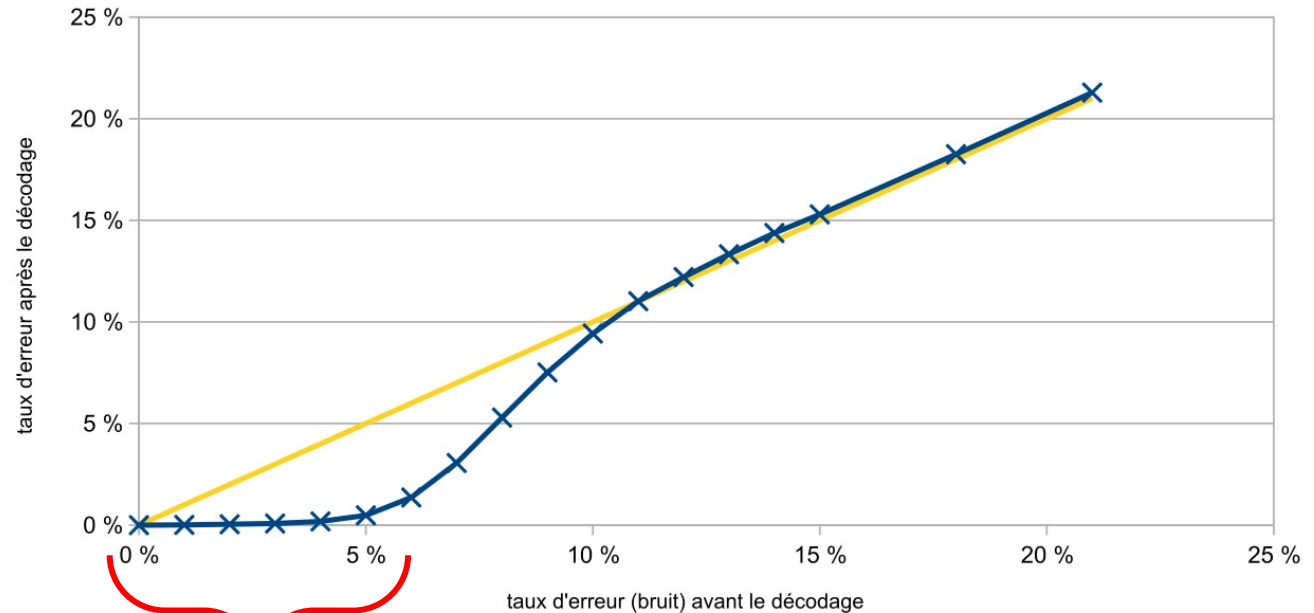


4) Complexités et paramètres

Capacité de correction :

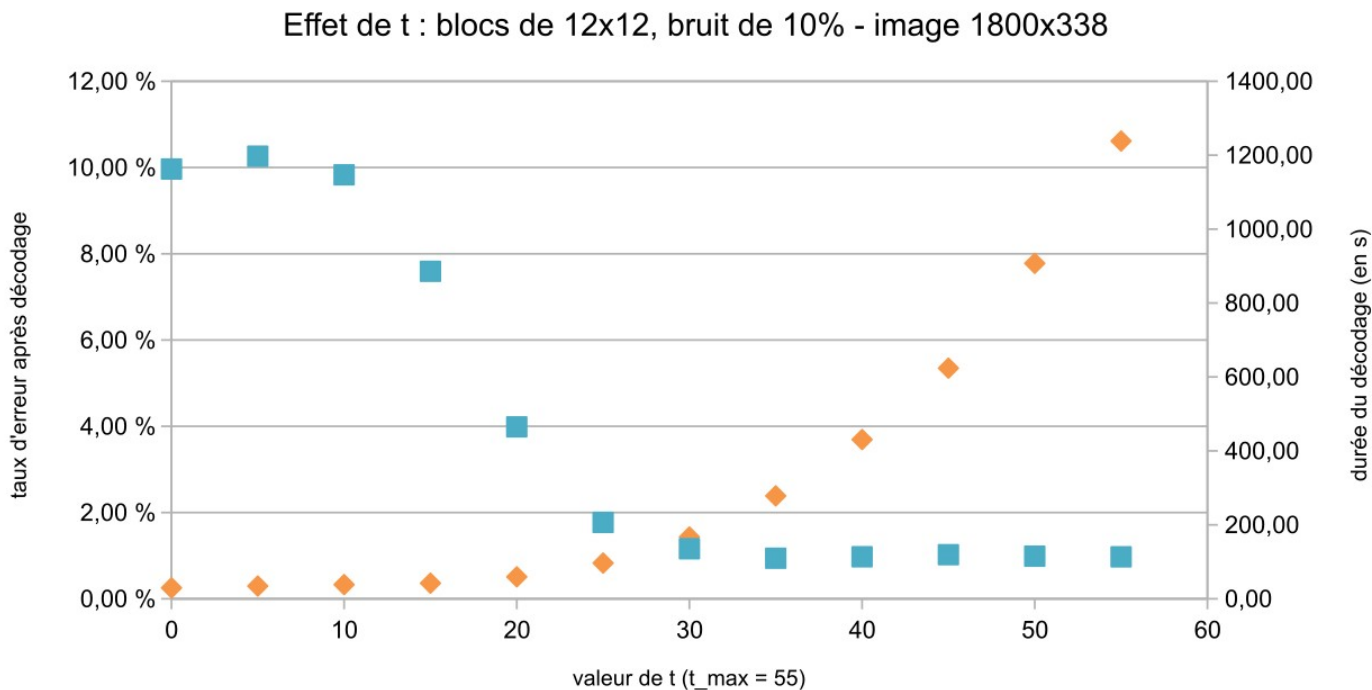
$$\frac{2t}{3xy} = \frac{2 \times 19}{3 \times 12 \times 18} \approx 0,06$$

Taux d'erreur : blocs de 12x18, t = 19 (sur 19) - image 1041x462

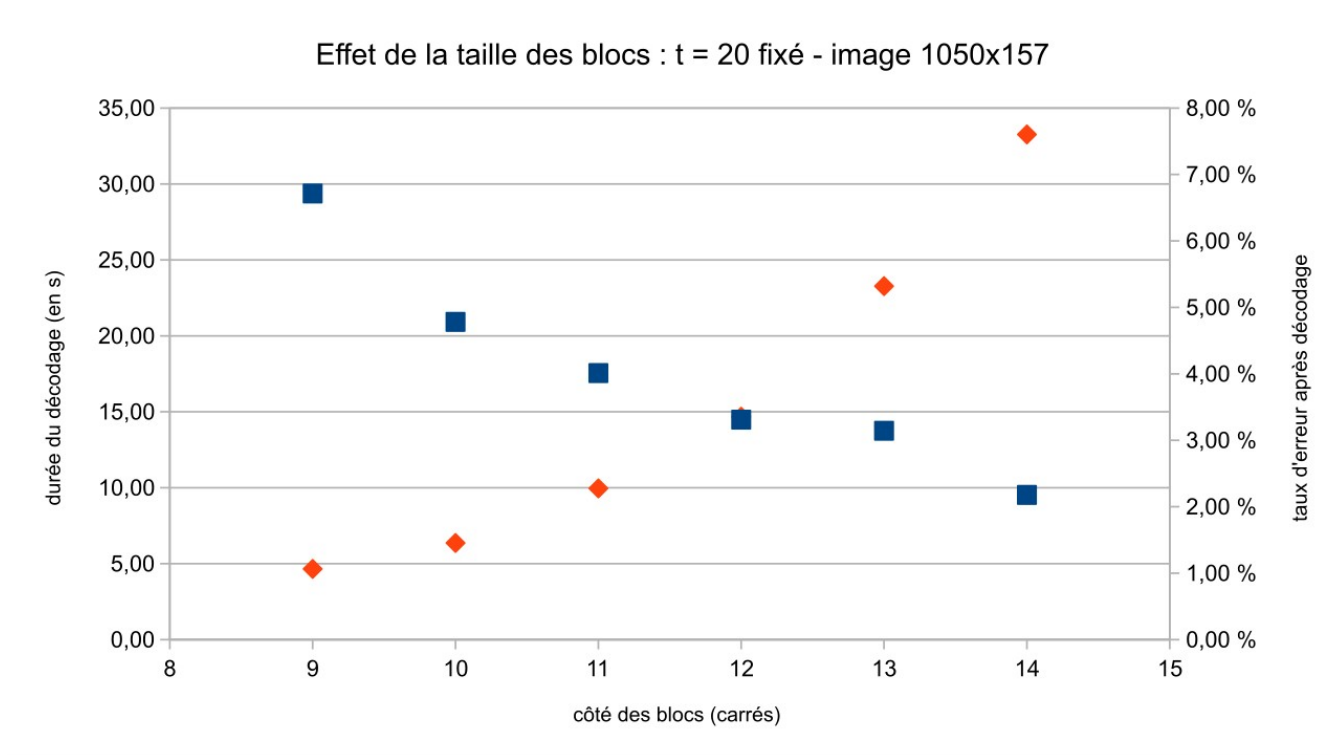


Zone optimale qui dépend de x, y et t

4) Complexités et paramètres



4) Complexités et paramètres



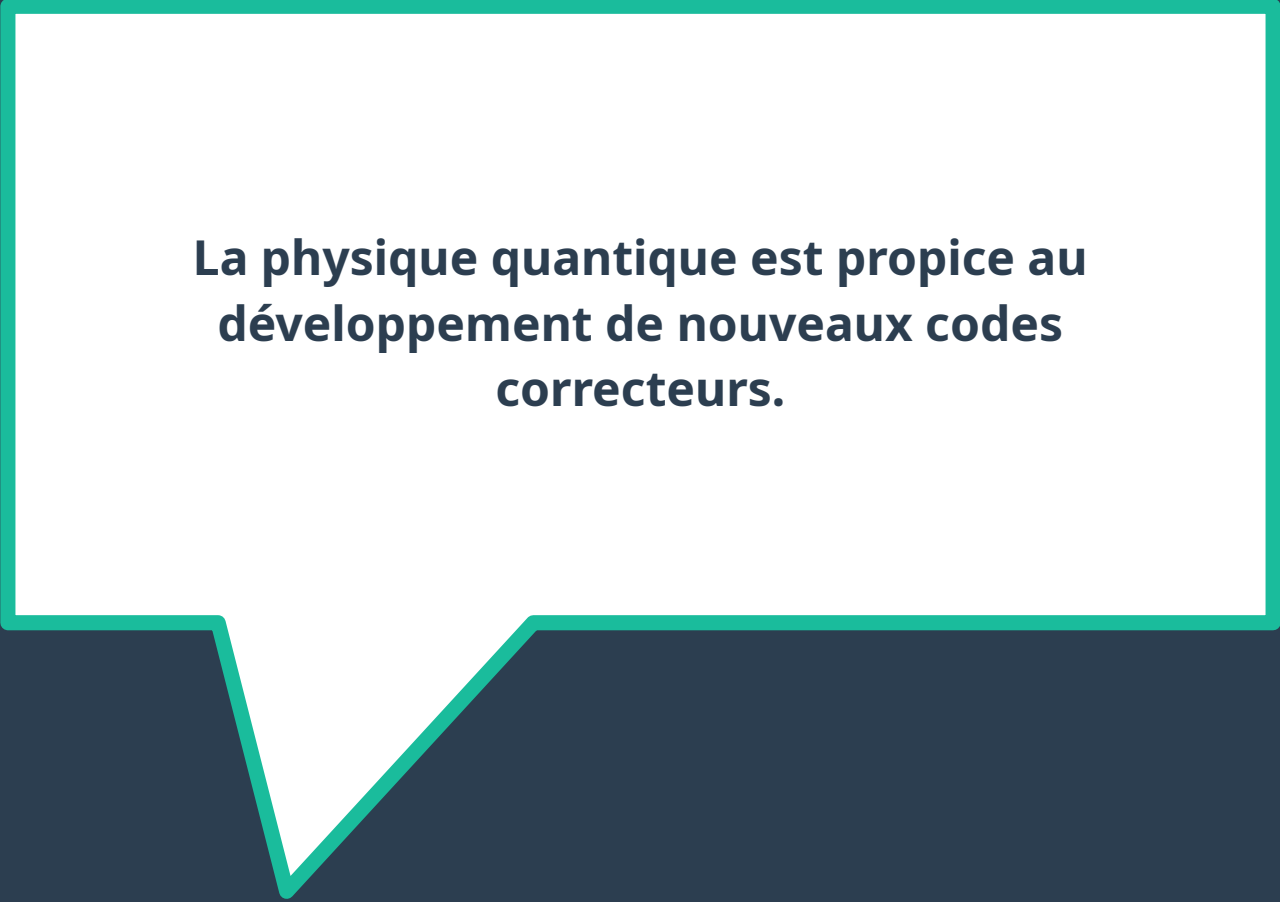
**Conclusion
&
perspectives**

Il y aura toujours des pixels erronés si le bruit est trop grand.

Cependant, le bruit atteint rarement 10 % mais est de l'ordre de 0,001 % = 10^{-5} pour des communications sans fils standards*.

**Il y a eu de nouveaux codes correcteurs
depuis Reed-Solomon :**

- les Low-Density Parity Check (LDPC)**
- les turbo codes**



**La physique quantique est propice au
développement de nouveaux codes
correcteurs.**



Merci de votre attention

Annexe

Le corps $\mathbb{F}_{256}$

$$X^8 + X^7 + X^2 + X + 1 = 110000111 \quad (\text{corps de caractéristique 2})$$

$$\begin{aligned} \overline{X^5 + 1} \times \overline{X^3 + 1} &= \overline{X^8 + X^5 + X^3 + 1} \\ &= \overline{-X^7 - X^2 - X - 1 + X^5 + X^3 + 1} = \overline{X^7 + X^5 + X^3 + X^2 + X} \\ &= \overline{X^{87}} \times \overline{X^{205}} = \overline{X^{292}} = \overline{X^{37}} = \overline{X^7 + X^5 + X^3 + X^2 + X} \end{aligned}$$

$\mathbb{F}_{256}^*$ est un groupe cyclique. On note $\alpha = \overline{X}$ qui engendre $\mathbb{F}_{256}^*$



Gorenstein-Zierler : syndromes

Transmission et décodage : on reçoit $D \in \mathbb{F}_{256}[X]$, $\deg(D) \leq 255$

Syndromes : $\forall j \in \llbracket 1, 2t \rrbracket, S_j = D(\alpha^j)$

⚠ Si $\forall j \in \llbracket 1, 2t \rrbracket, S_j = 0$,

$$D = R \times \prod_{j=1}^{2t} (X - \alpha^j) = RG \quad \text{et} \quad D - C = G(R - Q) \equiv 0 [G]$$

On ne peut pas corriger dans ce cas.



```

7
6
5 #####
4 # ENCODAGE INDIVIDUEL #
3 #####
2
1 def encodage_v2(message, k, t, results, i):
H 194 | -- 6 lines: P = [F256(0) for i in range(2 * t + 1)].....
1
2 def evaluation(P, a):
3 +-- 4 lines: s = F256(0).....
4
5
6 #####
7 # DECODAGE INDIVIDUEL #
8 #####
9
10 def identite(n):
11 +-- 4 lines: mat = np.full((n, n), F256(0), dtype=F256).....
12
13 def algo_du_pivot(mat):
14 +-- 35 lines: cop = mat.copy().....
15
16 def syndromes(recu, t):
17 |     return [evaluation(recu, table_exp[i]) for i in range(2 * t)]
18
19 def det_poly_correcteur(t, synds):
20 +-- 11 lines: for nu in range(t, 0, -1):.....
21
22 def det_ir_xr(poly_lambda):
23 +-- 6 lines: ir, xr = [], [].....
24
25 def det_E(ir, xr, synds, k, t):
26 +-- 15 lines: nu = len(xr).....
27
28 def decodage_v2(recu, k, t, results, i):
29 +-- 15 lines: synds = syndromes(recu, t).....
30
31

```

```

7 #####
6 # MULTIPROCESS #
5 #####
4
3 # Execute sur chaque bloc une fonction f qui respecte le bon format
2 # Utile pour l'encodage et le décodage
1 def multiprocess(blocs, f, t, n_process, silent):
321     n_blocs = len(blocs)
1     blocs_traites = [None] * n_blocs
2     # Gestion des processus en parallèle
3     process = [None] * n_process
4     manager = Manager()
5     results = manager.list([None] * n_process)
6     coords_list = [None] * n_process
7
8     for g in range(n_blocs // n_process):
9         if g == 0:
10             debut = time.time()
11             # Lancement de chaque processus
12             for i in range(n_process):
13 --- 5 lines: empla = n_process * g + i.....
14             for i in range(n_process):
15 --- 4 lines: process[i].join().....
16             if g == 0:
17                 fin = time.time()
18                 if not silent:
19                     print("Temps estimé :", int((fin - debut) * (n_blocs // n_process)), "secondes.")
20
21     # S'il reste quelques processus on les fait (n_process peut ne pas diviser n_blocs)
22     m = n_blocs % n_process
23     for g in range(m):
24 --- 10 lines: for i in range(m):.....
25
26     return blocs_traites

```

```

main.py x
14 def encodage_v1(u):
13     k = u.shape[0]
12     a = np.zeros(255, dtype=F256)
11     for i in range(255):
10         c = F256(0)
9         for j in range(k):
8             c += u[j] * table_exp[(i * j) % 255]
7         a[i] = c
6     return a
5
4 def combi_random(jusqua, k, iter_max):
H 3     for i in range(iter_max):      # "i" is not accessed
2         yield random.sample(range(jusqua), k)
1
718 # Pour la suite, il est nécessaire que decodage ne renvoie pas de données
1 # mais modifie une liste en argument à l'emplacement prévu
2 def decodage_v1(w, k, lim_pointe, lim_max, liste, emplacement):
3     udict = {}
4     # Pour chaque façon de prendre k équations parmi les 255
5     for kcombi in combi_random(255, k, lim_max):
6         w_k = np.array([w[x] for x in kcombi])
7         matrice_systeme = np.zeros((k, k), dtype=F256)
8         for i in range(len(kcombi)):
9             for puissance in range(k):
10                 matrice_systeme[i, puissance] = table_exp[(kcombi[i] * puissance) % 255]
11         u = algo_du_pivot(matrice_systeme) @ w_k
12         # On convertit u en un objet "hashable"
13         hash_u = tuple([int(i) for i in u])
14         if hash_u not in udict:
15             udict[hash_u] = 0
16         udict[hash_u] += 1
17         # Dès que l'on a plus de "limite" équations menant au même vecteur on s'arrête
18         max_compte = 0
19         max_u = None
20         for u in udict:
21             if udict[u] > max_compte:
22                 max_compte = udict[u]
23                 max_u = u
24         if max_compte >= lim_pointe:
25             liste[emplacement] = np.array(max_u)
26         return

```